
Wrong Family, Right Pattern: Fidelity-Based Model Fitting for Spatial Point Processes Near Complete Spatial Randomness

Anonymous Author
Anonymous Institution

Abstract

[TBD: abstract; write last. A pre-skeleton draft exists in `docs/aistats.outline.tex` and will need re-aligning with the section structure below.]

1 BACKGROUND

1.1 Spatial Point Processes and Second-Order Summary Statistics

- Define a point process X on the unit window $W = [0, 1]^2$ (the only window in the code), with expected count $\bar{n}$. Also define stationarity and isotropy, and CSR (homogeneous Poisson) as the null.
- Ripley's K and $L(r) = \sqrt{K(r)/\pi}$. The code works with $\sqrt{n}(L(r) - r)$ and the isotropic edge-corrected $\hat{K}$ (`src/cloudforger/classical/lfunction.py`).
- Pair correlation $g(r) = K'(r)/2\pi r$: defined only. It appears in the closed forms (Matérn I/II, LGCP; Table 1) and as a limit of the kernel score (Section 4). It is never estimated as a feature.
- Other summaries the models actually use: empty-space F , nearest-neighbour G , and $J = (1 - G)/(1 - F)$ (`src/cloudforger/classical/fgj.py`).
- [NEEDS INPUT: introduce the departure-from-CSR statistic $\tilde{\delta}$ (a studentised global envelope on L , `docs/01.departure.tex`) here? It stratifies the score power check (T8) and yields the hard-core boundary in T5.]

1.2 Persistent Homology (Optional — To Be Decided)

- **Status: optional, not final.** Decide after the PH ablation (T10).
- Persistence diagrams (H_0, H_1 ; essential classes dropped) of three filtrations: Vietoris–Rips, alpha (diameter scale), and DTM-Rips with $k \in \{5, 10, 15, 20\}$, $q = 2$ (`src/cloudforger/featurization/filtrations.py`, `configs/featurization/config.yaml`).
- Coordinates are rescaled by $\sqrt{n}$ (mean-spacing units), so diagrams are comparable across $\bar{n}$.
- Dependency: if this subsection is cut, Section 3.1 still needs a minimal diagram definition, because the frozen model (`hgb_classical_ph`) uses PH summaries.

1.3 Processes

- Eight families, as implemented in `src/cloudforger/simulation/{families, processes}.py`. Give one line each and point to Table 1 for K .
- Neyman–Scott cluster processes: Thomas, nested (two-level) Thomas, and ring (offspring on a circle of radius ρ with radial jitter σ). Cox: LGCP with exponential covariance. Hard core: Matérn I and II. Plus Poisson.
- Generalised Baddeley–Silverman cell process: $K(r) = \pi r^2$ exactly, so it is invisible to every second-order statistic but is not Poisson. This motivates Sections 1.2 and 5.
- Note which families approach others as a limit (ring as $\sigma/\rho \rightarrow 1$, and nested with a smeared outer level, both tend to Thomas). This matters for the confusions in Section 5.

Table 1: [PLACEHOLDER — The eight process families: parameters, second-order structure, closed-form K and the coordinate that locates each family’s near-CSR regime.]

Family	Parameters	#	Structure	Stat./iso.	$K(r) - \pi r^2$	Regime coord.
Poisson	$\bar{n}$	1	CSR	yes / yes	0	—
Thomas	κ, μ, σ	3	clustered	yes / yes	$\kappa^{-1}(1 - e^{-r^2/4\sigma^2})$	$\omega = \sigma\sqrt{\kappa}$
Nested	$\kappa, \mu_1, \mu_2, \sigma_1, \sigma_2$	5	clustered, 2 scales	yes / yes	$\kappa^{-1}[\mu_1^{-1}(1 - e^{-r^2/4\sigma_2^2}) + (1 - e^{-r^2/4(\sigma_1^2+\sigma_2^2)})]$	$\omega_{\text{in}} = \sigma_2\sqrt{\kappa\mu_1}$
Ring	$\kappa, \mu, \rho, \sigma$	4	clustered, annular	yes / yes	$\kappa^{-1}F_D(r; \rho, \sigma)$; F_D = CDF of the offspring–offspring distance (1-D integral)	$\omega = \rho\sqrt{\kappa}$
LGCP	$\mu_{\log}, \sigma^2, s$	3	clustered (Cox)	yes / yes [†]	$2\pi \sum_{j \geq 1} \frac{(\sigma^2)^j}{j!} \left(\frac{s}{j}\right)^2 [1 - (1 + \frac{jr}{s})e^{-jr/s}]$	$\zeta = (e^{\sigma^2} - 1)s\bar{n}$
Matérn II	R, λ_p	2	regular (hard core)	yes / yes	$2\pi \int_0^r t g(t) dt - \pi r^2$; g closed form, piecewise on $[0, R), [R, 2R), [2R, \infty)$	$\zeta = R\bar{n}$
Matérn I	R, λ_p	2	regular (hard core)	yes / yes	as Matérn II, with $g(r) = \exp(\lambda_p b(0, R) \cap b(r, R))$ on $[R, 2R)$	$\zeta = R\bar{n}$
Cell	$\bar{n}, k$	2	second-order CSR	yes / no [‡]	0 (exactly)	none (U-shaped in k)

[†]Simulated piecewise-constant on an $M \times M$ grid (`src/cloudforger/simulation/lgcp-grid.py`). [‡]Randomly shifted, axis-aligned grid of cells.

2 DATA GENERATION

Sweep coordinates.

- Random design, not a grid. Each θ is an i.i.d. draw from a constrained log-uniform prior on its own seed stream, so the bank is reproducible and can be extended without moving existing draws (`src/cloudforger/simulation/{bank,seeding}.py`).
- Every family first draws a count budget $\bar{n} \sim \text{log-uniform}[75, 950]$. The last parameter then follows from conservation of the mean count.
- Cluster families: two independent dials, richness (points per cluster) and smearing $\omega \in [0.2, 8]$. Nested adds the scale ratio σ_1/σ_2 ; ring adds the jitter σ/ρ . LGCP, Matérn and cell have their own dials. Full ranges are in Table 2 (`configs/simulation/config.yaml, rules`).

Constraints.

- Count-variance bound $\text{cv}(n) = \text{sd}(n)/\bar{n} \leq 0.30$, computed in closed form from K . It is *inverted into the support* rather than rejected, so the richness dial is not reweighted (`docs/02_simulation.tex`).
- Degeneracy guards: at least 10 clusters in the window; minimum points per cluster, meta-cluster and ring; Matérn packing caps (Matérn I cannot exceed $1/e$); LGCP range resolvable and within the circulant-embedding limit; a σ cost guard.

- Realised counts are conditioned on $n \in [20, 2000]$. Parents are simulated on a window buffered by 5 displacement s.d.s, to avoid edge loss.

Data volume.

- 8 families $\times$ 50 000 θ $\times$ 2 independent replicates = 800 000 patterns (`data/bank/FAMILY/manifest.csv`).
- Split by θ index, the same for every family, model and seed, with both replicates in the same split: 37 000 / 1 000 / 12 000 θ per family (`src/cloudforger/simulation/split.py`).
- The test block is large on purpose: near-CSR regime cells need enough θ for bootstrap intervals. The end-to-end evaluation subsets are drawn from it (Section 4).

3 THE MODEL

- Pipeline = the posterior factorisation $p(f, \theta | x) = p(f | x)p(\theta | x, f)$. One 8-way classifier, whose $P(\text{Poisson} | x)$ serves as the CSR filter. A Poisson call gets $\hat{n} = n$ (the exact MLE); any other call goes to that family’s estimator of $\log \theta$ (`oneshot/README.md`).
- Any model can serve as classifier or estimator; every choice is a config key (`oneshot/configs/default.yaml`).

Table 2: [PLACEHOLDER — Simulation design: the dials swept per family, their ranges, and the number of parameter draws, replicates and patterns in each split.]

Family	Drawn (all log-uniform; * = on the feasible sub-range)	Derived	θ	Reps	Patterns	Train / val / test
Poisson	$\bar{n}$	—	50 000	2	100 000	74 000 / 2 000 / 24 000
Thomas	$\bar{n}; \mu \in [1, \bar{n}/10]; \omega \in [0.2, 8]^*$	$\kappa = \bar{n}/\mu, \sigma = \omega/\sqrt{\kappa}$	50 000	2	100 000	74 000 / 2 000 / 24 000
Nested	$\bar{n}; \mu_2 \in [1, \bar{n}/10]; \mu_1 \in [\max(1, 2/\mu_2), \bar{n}/10\mu_2]; \sigma_1/\sigma_2 \in [2.49, 10]; \omega = \sigma_1\sqrt{\kappa} \in [0.2, 8]^*$	$\kappa = \bar{n}/\mu_1\mu_2$	50 000	2	100 000	74 000 / 2 000 / 24 000
Ring	$\bar{n}; \mu \in [3, \bar{n}/10]; \sigma/\rho \in [0.05, 1]; \omega = \rho\sqrt{\kappa} \in [0.2, 8]^*$	$\kappa = \bar{n}/\mu$	50 000	2	100 000	74 000 / 2 000 / 24 000
LGCP	$\bar{n}; s \in [0.10/\sqrt{\bar{n}}, 0.20]; \sigma^2 \in [0.05, \min(10, \text{cv cap})]$	$\mu_{\log} = \log \bar{n} - \sigma^2/2; \text{grid } M$	50 000	2	100 000	74 000 / 2 000 / 24 000
Matérn II	$\bar{n}; R\sqrt{\bar{n}} \in [0.05, \sqrt{0.95/\pi}]$	λ_p (exact inversion)	50 000	2	100 000	74 000 / 2 000 / 24 000
Matérn I	$\bar{n}; R\sqrt{\bar{n}} \in [0.05, \sqrt{0.35/\pi}]$	λ_p (Lambert W)	50 000	2	100 000	74 000 / 2 000 / 24 000
Cell	$\bar{n}; k \in \{2, \dots, 30\}$	—	50 000	2	100 000	74 000 / 2 000 / 24 000
Total			400 000		800 000	592 000 / 16 000 / 192 000

[F1 placeholder: example realizations, two per family]

Figure 1: [PLACEHOLDER — Example realizations of each family, one strongly structured and one near CSR.]

3.1 Vectorizations

- Classical vector (111 columns): $\sqrt{n}(L(r) - r)$ on a fixed radius axis, plus L, F, G, J on the $u = r\sqrt{n}$ axis, log-spaced samples, plus $\log n$ (`cascade/features.py`).
- PH summary vector (105 columns per filtration): Betti curves and lifetime / birth / death statistics for H_0, H_1 (`src/cloudforger/vectorization/summaries.py`). Alpha and DTM_{10} in the frozen model; six filtrations in `hgb_classical_ph6`.
- Network inputs: L, F, G, J curves stacked as channels of one 1-D signal; persistence images per homology dimension (bounds fitted on train); PersLay, a learned permutation-invariant diagram layer; and curves plus alpha images fused in one network. $\log n$ is a covariate everywhere.
- [NEEDS INPUT: include the raw-point DeepSets / GNN runs (`extensions/learned/`; untracked, smoke scale only)?]

3.2 Architectures

- Gradient-boosted trees (`HistGradientBoosting`, fixed 400 iterations) on concatenated feature tables: four input sets, including the frozen `hgb_classical_ph`.
- Linear: standardised logistic regression on the classical vector (classifier only).
- PHNet (`src/cloudforger/training/model.py`): one CNN or PersLay encoder per input, embeddings concatenated with $\log n$, then an MLP head; AdamW with early stopping on validation loss.
- References: minimum contrast on K with penalised-contrast model selection (`oneshot/mincontrast.py`), and oracle routing to the true family.
- [NEEDS INPUT: headline estimator assignment: `hgb_classical_ph` for every family (the “ph” pipeline, frozen in `docs/aistats-outline.tex`), or the per-family best on validation (“best”)?]

Table 3: [PLACEHOLDER — Models compared: learner, input representation, tasks, parameter count and training cost.]

Model	Learner	Input (dim)	Task	# params	Clf. time (s)	Est. time (s)	Hardware /
<code>hgb_classical</code>	trees	classical (111)	C, E	[TBD]	515	[TBD]	CPU; 400 it
<code>hgb_ph</code>	trees	PH alpha + DTM ₁₀ (210)	C, E	[TBD]	154	[TBD]	CPU; 400 it
<code>hgb_classical_ph</code>	trees	classical + PH (321)	C, E	[TBD]	202	[TBD]	CPU; 400 it
<code>hgb_classical_ph6</code>	trees	classical + PH, 6 filtr. (741)	C, E	[TBD]	514	[TBD]	CPU; 400 it
<code>linear_classical</code>	linear	classical (111)	C	[TBD]	126	—	CPU
<code>nn_curves</code>	PHNet	L, F, G, J curves	C, E	[TBD]	1926	[TBD]	GPU; ≤ 200
<code>pi_alpha</code>	PHNet	pers. images, alpha	C, E	[TBD]	2432	[TBD]	GPU; ≤ 200
<code>pi_rips</code>	PHNet	pers. images, Rips	C, E	[TBD]	1708	[TBD]	GPU; ≤ 200
<code>perslay_alpha</code>	PHNet	PersLay, alpha	C, E	[TBD]	1453	[TBD]	GPU; ≤ 200
<code>perslay_dtm10</code>	PHNet	PersLay, DTM ₁₀	C, E	[TBD]	2510	[TBD]	GPU; ≤ 200
<code>fusion_curves_alpha</code>	PHNet	curves + alpha images	C, E	[TBD]	2349	[TBD]	GPU; ≤ 200
min. contrast	—	$\hat{K}$	C, E	0 (tuned on val)	[TBD]	[TBD]	CPU array
oracle	—	true family	C	—	—	—	—

[F2 placeholder: point cloud $\rightarrow$ vectorization $\rightarrow$ classifier / estimator $\rightarrow$ score]

Figure 2: [PLACEHOLDER — The pipeline, from point cloud to fitted model to end-to-end score.]

4 THE SCORE

- Training losses, per stage (`oneshot/learners.py`, `cascade/nn.py`). Classifiers: log loss, equal-prior sample weights (trees) or class-weighted cross-entropy (PHNet). Estimators: squared error on $\log \theta$ (trees) or MSE on z-scored $\log \theta$ (PHNet).
- Component metrics (`oneshot/compare.py`): accuracy, NLL, per-family recall and detection (not called Poisson), and estimator error $\text{RMSE}(\log \theta)/\text{s.d.}$ averaged over a family’s targets.
- End-to-end score, never optimised directly: a kernel score on local configurations of radius $R = 1.5$ mean spacings (Eq. 1). It is strictly proper for the law of radius- R configurations (`cascade/scoring/local_kernel.py`, `cascade/scoring/scores.tex`).
- Protocol: fit on replicate 0 and score on the independent replicate 1; the CSR reference is Poisson at $\bar{n} = n$; skill is reported only where the gain is resolved (> 2 s.e.). Dawid–Sebastiani is a secondary score (proper, not strictly). Evaluation clouds are stratified test subsets (`oneshot/configs/default.yaml`, `evaluation`).
- Why a new score: justify it with the power

Table 4: [PLACEHOLDER — Power of candidate end-to-end scores to separate the true model from CSR, by family and departure bin.]

check (T8), including the negative result for the Wasserstein-1 energy score.

$$S(P, x) = \frac{1}{2} \mathbb{E} k(\Phi, \Phi') - \frac{1}{m} \sum_{i=1}^m \mathbb{E} k(\phi_i, \Phi) \quad (1)$$

[PLACEHOLDER: local configuration ϕ_i , grid embedding, kernel w

$$\text{regret} = S(\hat{P}) - S(P_\star), \quad \text{gain} = S(P_{\text{CSR}}) - S(P_\star), \quad \text{skill} = 1 - \frac{\sum \text{regret}}{\sum \text{gain}} \quad (2)$$

$$\Delta_{\text{CSR}} = S(P_{\text{CSR}}) - S(\hat{P}) = \text{gain} - \text{regret} \quad (3)$$

$$S_{\text{DS}}(P, x) = \log \det \Sigma_P + (T(x) - \mu_P)^\top \Sigma_P^{-1} (T(x) - \mu_P) \quad (4)$$

5 RESULTS

5.1 Classification Results

- 11 learned classifiers plus minimum-contrast selection, on the 192 000 test patterns with an equal prior. Report 95% bootstrap intervals over test θ , and paired differences against `hgb_classical`.

[F3 placeholder: 8×8 confusion matrix]

Figure 3: [PLACEHOLDER — Confusion matrix of the frozen classifier on the test split.]

- Accuracy pooled and in regime ($\tau = 0.5, 0.9$), with per-family recall and detection. What to state: whether the top classifiers can be told apart at all.
- Confusions by type: calls to Poisson (Section 5.2) versus confusions between families that coincide over part of their parameter space (Section 1.3).

5.2 Misclassification-Regime Analysis

- For each non-Poisson family, take $P(\text{called Poisson} \mid \theta) = 1 - \text{detected from the reference classifier's out-of-fold predictions on train.}$
- Regime rule (`cascade/regime.py`): $u = \log x + a \log \bar{n}$ with a fitted by logistic regression, $P(\text{detected} \mid u)$ isotonic in u , and boundary $u^*(\tau)$. The result is a frozen rule $x \bar{n}^a \geq e^{u^*}$ on θ alone.
- Show that Poisson calls concentrate on one side of the boundary. Compare the fitted boundary with analytic candidates: the hard-core boundary from the $\tilde{\delta}$ derivation (`docs/01_departure_notes.md`); [NEEDS INPUT: no analytic derivation exists in the repo for the cluster families or LGCP].
- [NEEDS INPUT: reference classifier: config uses `hgb-classical`; the frozen headline classifier is `hgb-classical-ph`.]

5.3 “Poisson Is Good Enough”

- Trivial baseline: CSR at $\bar{n} = n$ (the Poisson MLE), scored with Eq. 1 on the same held-out replicate as the fit.
- Quantity: the score gap Δ_{CSR} (Eq. 4) and the oracle gain, per family and regime stratum (below 0.5 / 0.5–0.9 / above 0.9 of $P(\text{detected} \mid u)$; cell by k).
- Claim to test: outside the regime the gap and the gain are both indistinguishable from zero, and inside they are not. For structured clouds routed to Poisson, regret = gain, so their gain *is* the cost of the Poisson call.

- Caveat: power near CSR (T8). Separate “no gap” from “no detectable gap”.
- [NEEDS INPUT: include the training-filter negative result (`oneshot/results/default/train_filter/gains.csv`) here or in the appendix?]

5.4 Final Results

- Reserved for the headline: T7 and F5 aggregate classification, estimation and end-to-end fidelity, against minimum contrast (with inferred and with true family) and oracle routing.
- Supporting tables: parameter-estimation error (T9) and the PH ablation (T10, conditional on Section 1.2).
- [NEEDS INPUT: real-data demonstration and a `spatstat kppm` baseline, both listed as “must have” in `docs/aistats_outline.tex`; neither exists in the repo.]

References

References

- Adams et al. (2017). Persistence images. [NEEDS INPUT: full reference]
- Anai et al. (2020). DTM-based filtrations. [NEEDS INPUT: full reference]
- Carrière et al. (2020). PersLay. AISTATS. [NEEDS INPUT: full reference]
- Diggle (1983). Minimum contrast. [NEEDS INPUT: full reference]
- Myllymäki et al. (2017). Global envelope tests. [NEEDS INPUT: full reference]
- Vihrs et al. (2022). Neural summaries for point processes. [NEEDS INPUT: full reference]
- Waagepetersen and Guan (2009). *JRSS-B* 71(3). [NEEDS INPUT: full reference]

Checklist

1. For all models and algorithms presented, check if you include:
 - (a) A clear description of the mathematical setting, assumptions, algorithm, and/or model. [TBD]

Table 5: [PLACEHOLDER — Eight-way classification: accuracy overall and in regime, and recall per family, for every classifier and the minimum-contrast baseline.]

[F4 placeholder: P(called Poisson) over sweep coordinates, with boundaries]

Figure 4: [PLACEHOLDER — Probability of being classified as Poisson across each family’s parameter space, with the fitted and analytic regime boundaries.]

Table 6: [PLACEHOLDER — Per-family boundary of the regime in which patterns look like Poisson, fitted and near-closed-form, in terms of the sweep parameters.]

Family	x	Fitted rule	Near-closed form
Thomas	$\sigma\sqrt{\kappa}$	$x\tilde{n}^a \geq c_\tau$ [TBD]	[NEEDS INPUT:]
Nested	$\sigma_2\sqrt{\kappa\mu_1}$	[TBD]	[NEEDS INPUT:]
Ring	$\rho\sqrt{\kappa}$	[TBD]	[NEEDS INPUT:]
LGCP	$(e^{\sigma^2} - 1)s\tilde{n}$	[TBD]	[NEEDS INPUT:]
Matérn II	$R\tilde{n}$	[TBD]	from $\tilde{\delta}_{lo}$ [TBD]
Matérn I	$R\tilde{n}$	[TBD]	[TBD: same argument?]
Cell	—	none (U-shaped in k)	—

Table 7: [PLACEHOLDER — Score gap between the fitted model and a Poisson fit, inside versus outside each family’s regime.]

- (b) All the training details (e.g., data splits, hyperparameters, how they were chosen). [TBD]
- (c) A clear definition of the specific measure or statistics and error bars (e.g., with respect to the random seed after running experiments multiple times). [TBD]
- (d) A description of the computing infrastructure used. (e.g., type of GPUs, internal cluster, or cloud provider). [TBD: GPU model not recorded]
4. If you are using existing assets (e.g., code, data, models) or curating/releasing new assets, check if you include:
 - (a) Citations of the creator If your work uses existing assets. [TBD]
 - (b) The license information of the assets, if applicable. [TBD]
 - (c) New assets either in the supplemental material or as a URL, if applicable. [TBD]
 - (d) Information about consent from data providers/curators. [TBD]
 - (e) Discussion of sensible content if applicable, e.g., personally identifiable information or offensive content. [TBD]
2. For any theoretical claim, check if you include:
 - (a) Statements of the full set of assumptions of all theoretical results. [TBD]
 - (b) Complete proofs of all theoretical results. [TBD]
 - (c) Clear explanations of any assumptions. [TBD]
3. For all figures and tables that present empirical results, check if you include:
 - (a) The code, data, and instructions needed to reproduce the main experimental results (either in the supplemental material or as a URL). [TBD]
5. If you used crowdsourcing or conducted research with human subjects, check if you include:
 - (a) The full text of instructions given to participants and screenshots. [TBD]
 - (b) Descriptions of potential participant risks, with links to Institutional Review Board (IRB) approvals if applicable. [TBD]
 - (c) The estimated hourly wage paid to participants and the total amount spent on participant compensation. [TBD]

Table 8: [PLACEHOLDER — Main results: classification, parameter error and end-to-end fidelity of the pipeline against classical and oracle baselines.]

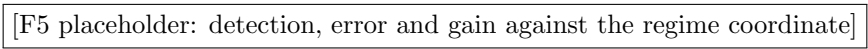


Figure 5: [PLACEHOLDER — Main results figure: detection, parameter error and the true model’s gain over CSR, against each family’s regime coordinate.]

Table 9: [PLACEHOLDER — Parameter-estimation error per family and estimator, overall and in regime, with minimum contrast.]

Table 10: [PLACEHOLDER — Paired effect of adding persistent-homology features on estimation, classification and end-to-end fidelity.]